



COMPSCI 130 S1 2022

Introduction to Software Fundamentals

Binary Trees – Introduction, Definition and Terminology

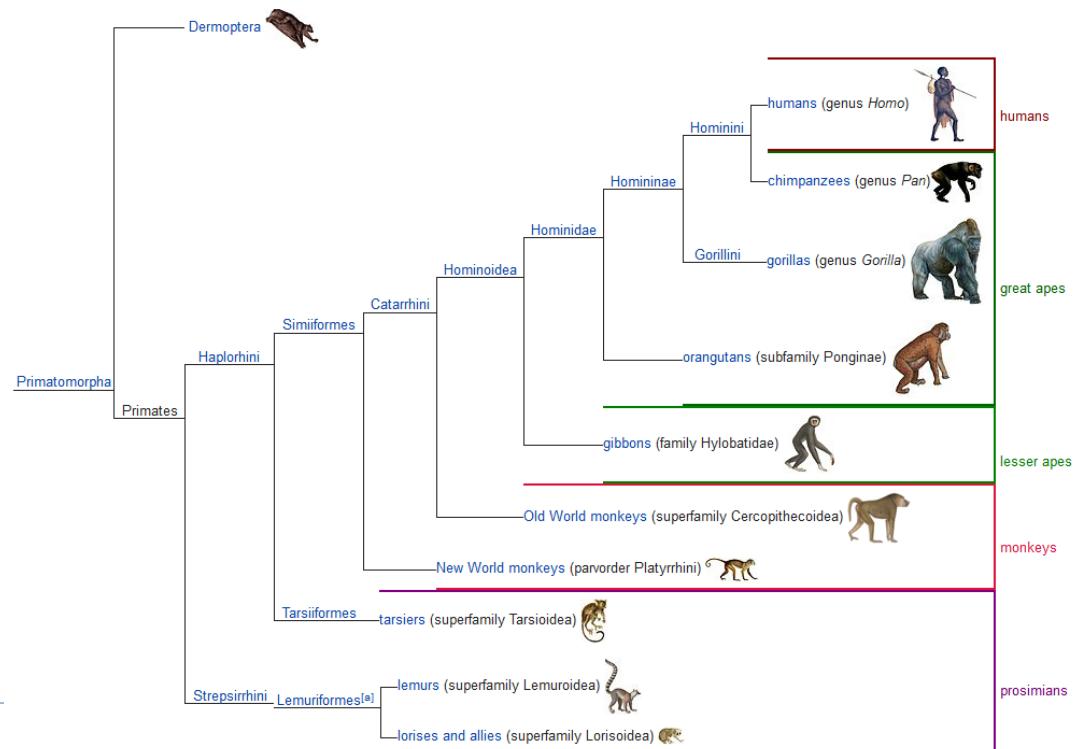


Agenda & Reading

- ▶ **Agenda**
 - ▶ What is a tree?
 - ▶ Applications of trees
 - ▶ Terminology
 - ▶ Recursive Definition

- ▶ **Online Coursebook:**
 - ▶ <https://onlinetextbook.auckland.ac.nz/19-binary-trees/>

- ▶ The data types we have studied so far have been **linear**
 - ▶ Stacks, Queues, Lists – in some sense these all have a first (top/front) value and a last (bottom/rear) value, and a linear collection of values in between
- ▶ But what about this classification of animals:





What is a Tree?

- ▶ A tree is an abstraction for a **hierarchical** structure. It is defined as a set of points called **nodes** and a set of lines called **edges** where an edge connects two distinct nodes.
- ▶ A tree has three properties:
 - ▶ One node is distinguished and called the **root**.
 - ▶ Every node n other than the root is connected by an edge to exactly one other node p closer to the root.
 - ▶ A tree is connected in the sense that if we start at any node n other than the root and move to the parent of n , continue to the parent of the parent of n , and so on, we eventually reach the root.



Applications of Trees

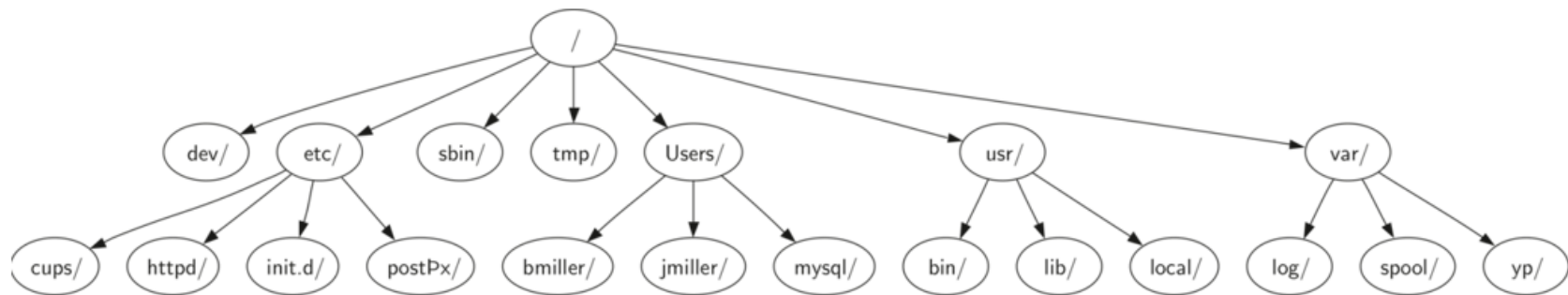
Examples and Applications of trees include:

- Family trees
- Directory structures (file system)
- Arithmetic expressions
- Game trees (finding winning positions in a game)
- Binary Space Partitioning (BSP) trees (finding visible objects in a scene)
- Search trees (finding all the paths back out of a maze)
- Quadtrees (for efficient terrain rendering)
- Octree (for fast collision detection, ray tracing etc.)
- Constructed Solid Geometry (CSG) objects
- Joint hierarchies (for skeletal animation of characters)
- etc.



Applications of Trees - File Systems

- ▶ A file system can be represented as a tree, with the top-most directory as the root.

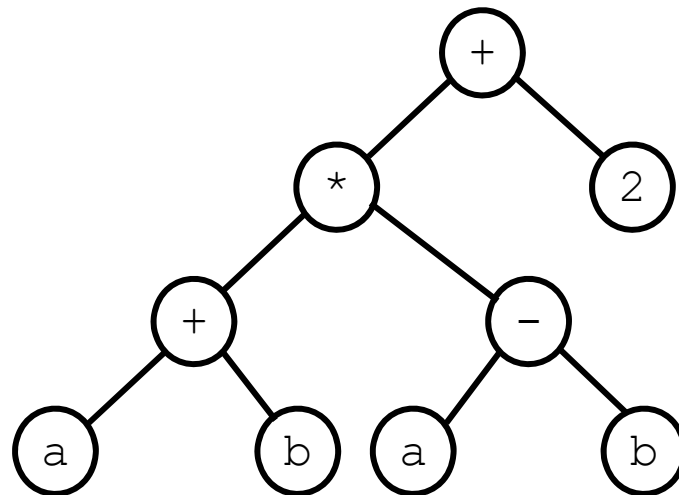




Applications of Trees – Arithmetic Expressions

- ▶ Arithmetic expressions can be expressed as trees
 - ▶ the leaves are the variables/values
 - ▶ the internal nodes are the operations

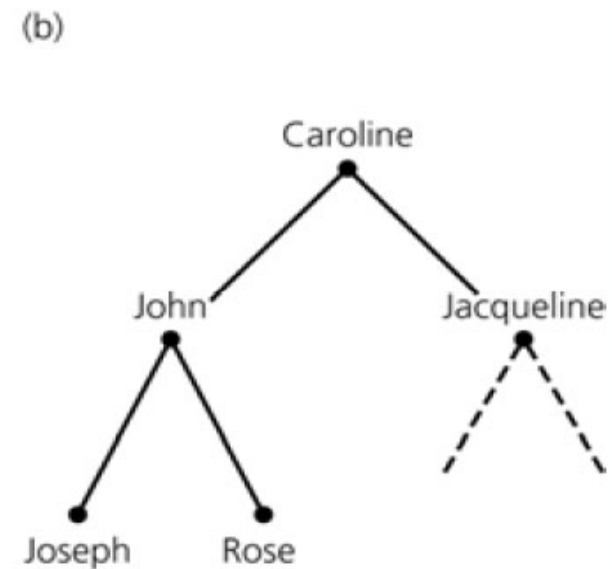
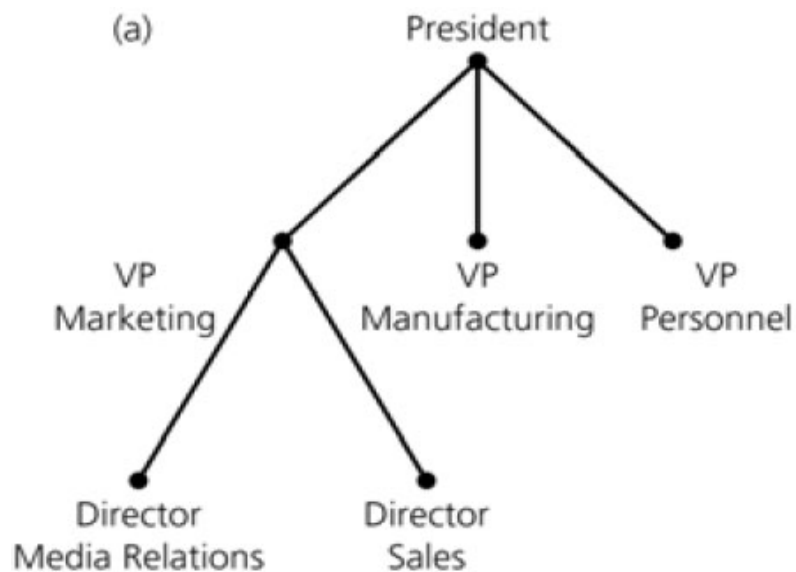
$$(a + b) * (a - b) + 2$$





Applications of Trees – Organizational Charts

- ▶ Organizational Charts and Family Trees are expressed as trees.

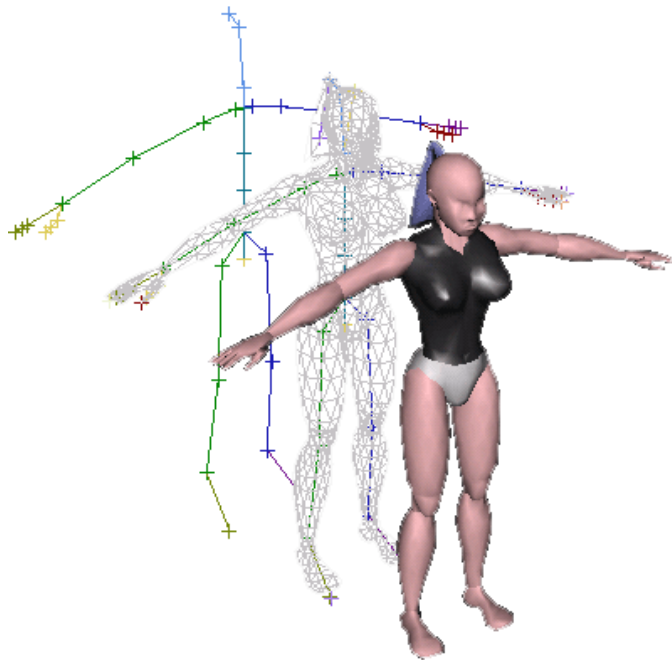




Applications of Trees – Bone Hierarchies

Trees can be used to represent bone hierarchies for skeletal Animation

- Movement around a joint (e.g., the elbow joint at the lower end of the upper arm) moves all child bones at the same time (e.g., the lower arm, hand and fingers)

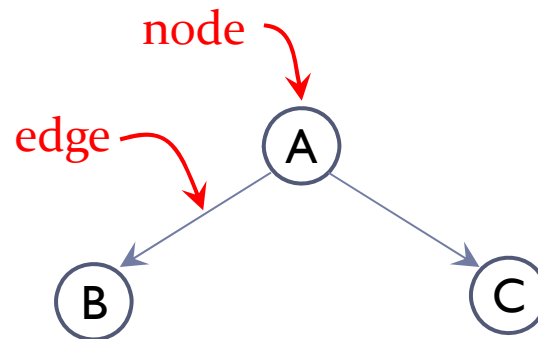




Terminology

Parts of a tree:

- ▶ Data objects (the circles) in a tree are called *nodes*.
- ▶ Links between nodes are called *edges*.
- ▶ An *empty tree* is a tree without any nodes.





Terminology

Relationships

- ▶ A is a **parent** of B and B is a **child** of A, if an edge points from A to B.
- ▶ A node can have only one parent.
- ▶ If two nodes have the same parent, they are **siblings**.
- ▶ **k-node tree** is a tree where every node has at most k children. With binary trees, a node can have at most 2 children.
- ▶ A **path** is a sequence of nodes $n_1, n_2, \dots, n_k$ such that n_1 is the parent of n_2 , n_2 is the parent of n_3 , and so on. The length of a path with k nodes is $k-1$. A path with a single node ($k = 1$) has a length of 0.
- ▶ A node a is an **ancestor** of a node d if there is a path from a to d . If node a is an ancestor of node d , then node d is a **descendant** of node a .



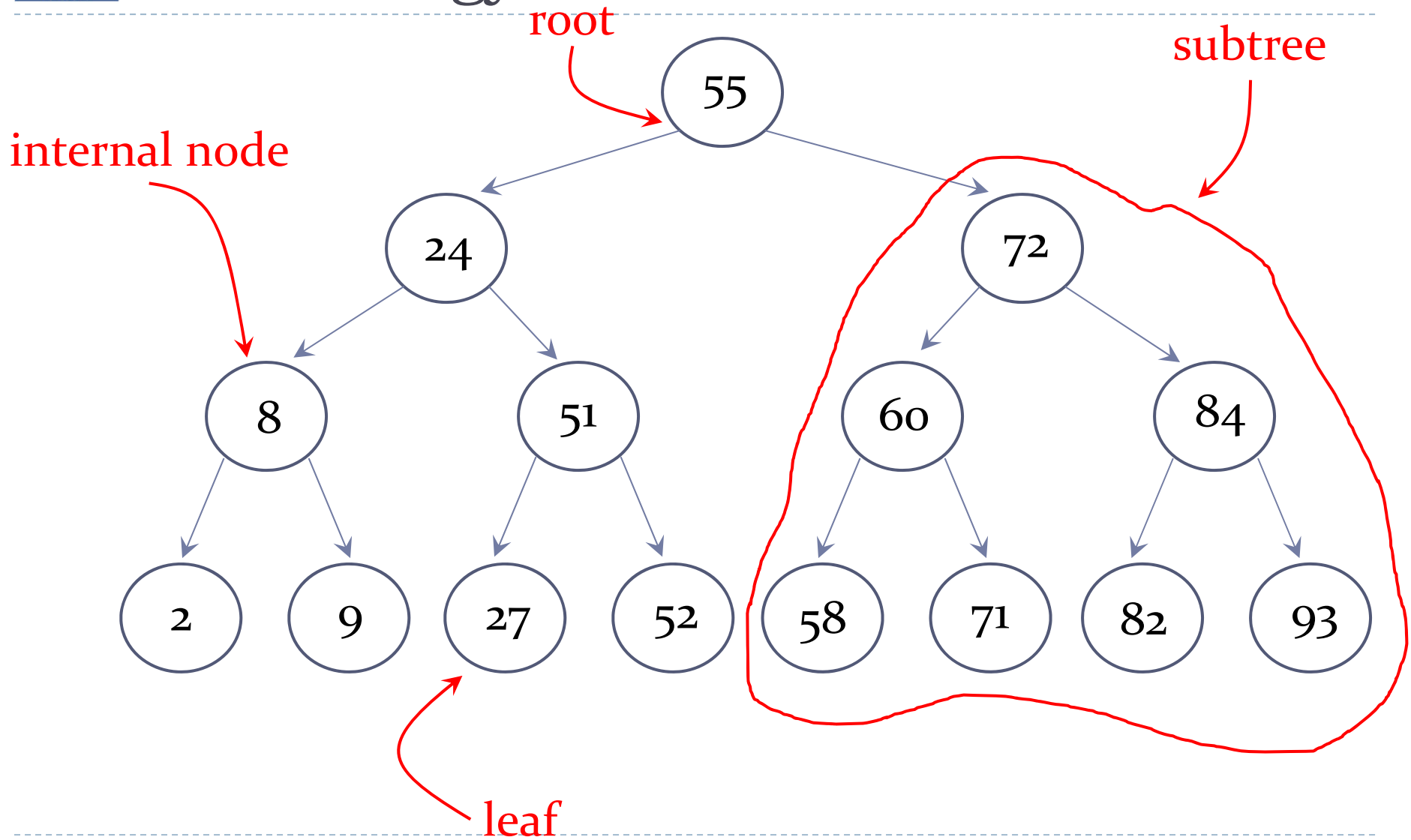
Terminology

Names for different tree nodes:

- ▶ A node without a parent is called the *root*.
- ▶ A node without children is called a *leaf*.
- ▶ Non-leaf nodes are also called *internal nodes*.
- ▶ A node and all its descendants form a *subtree*.



Terminology





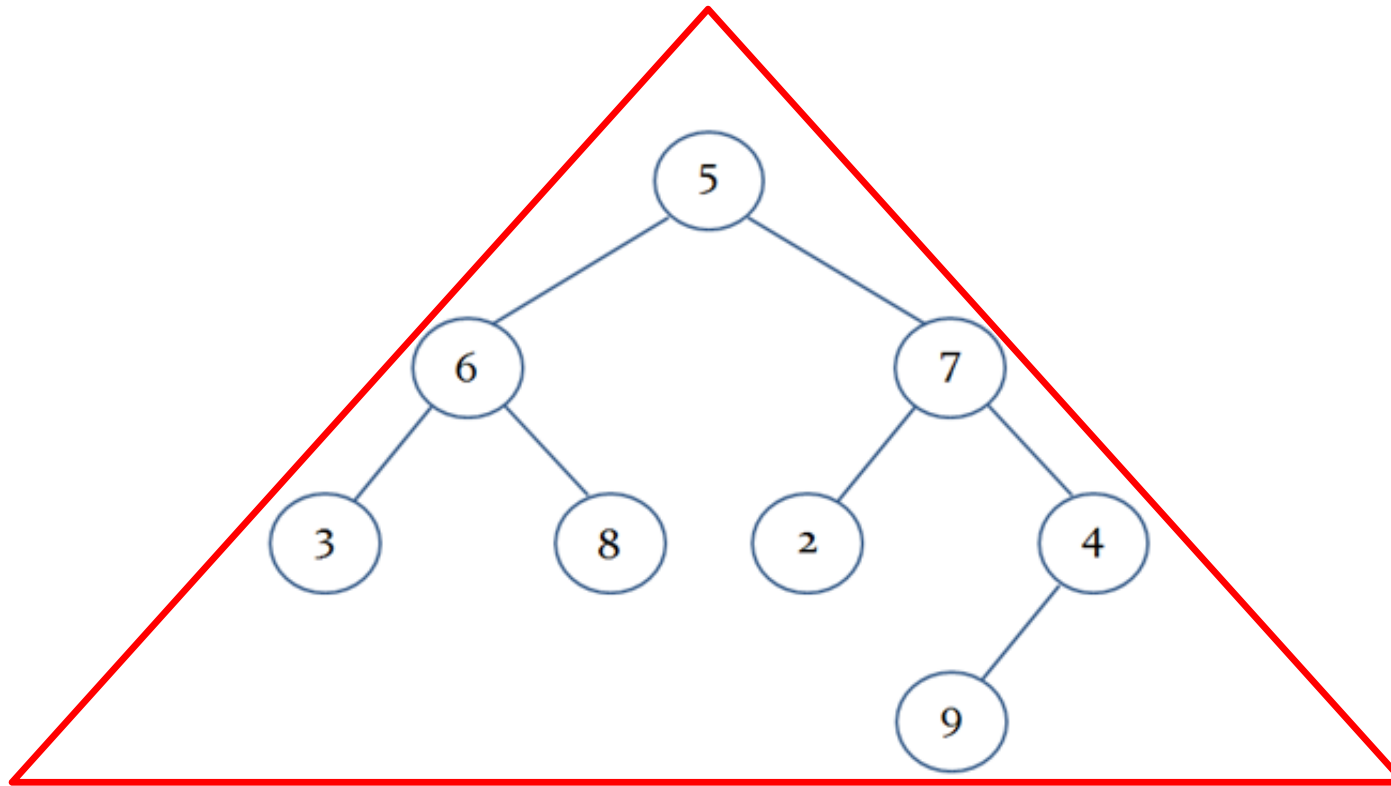
Recursive Definition of a Tree

- ▶ **Trees are recursive structures**
 - ▶ A tree is either empty, or it consists of a root node and zero or more subtrees, each of which is also a tree, connected to the root.



Example – Sum of Values

- Calculate the sum of values in the following tree:



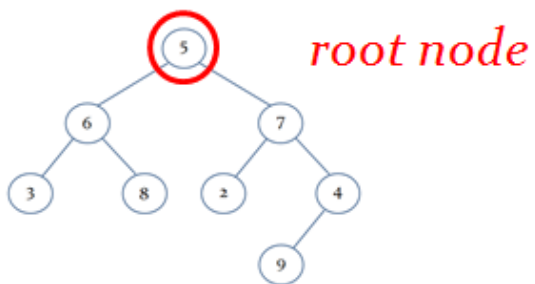
Sum of values in the tree



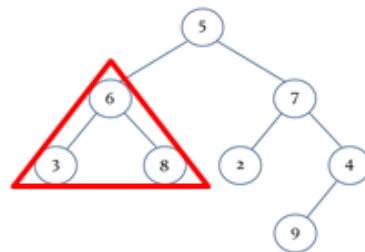
Example – Sum of Values

Sum of values in a tree

=

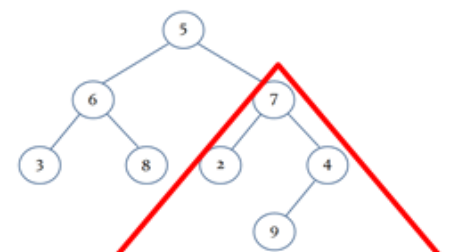


+



sum of left sub-tree

+

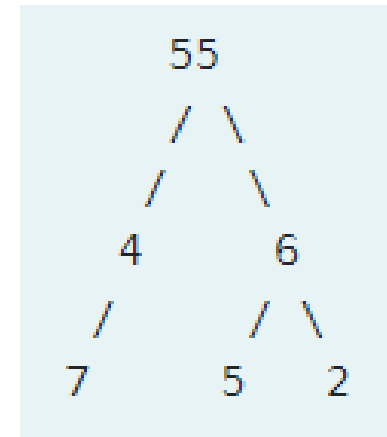


sum of right sub-tree



Exercise 1

► Consider the following binary tree:



- The number of leaf (external) nodes is:
- The number of internal nodes is:
- The height of the tree is:
- The sum of values of the tree is:
- Paths:
 - 55, 4, 7
 - 55
 - 55



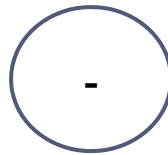
Exercise 2

► Consider the following infix expression:

► $((2 + 3) * 4) - 1$

► Convert the above infix expression to an expression tree.

look for the least
precedence operator





Exercise 2

- ▶ Consider the following infix notation:
 - ▶ $(5 * 3) + (6 - 2)$
- ▶ Convert the above infix expression into an expression tree.

